A first insider detector: Isolation Forest on wallet features

Stats C161/C261, guest lecture, Block 3. **Preview of Week 5**: the algorithm is explained properly there; here we run it end to end.

The data. `data/wallet_features.csv`: one row per wallet, for every wallet that bought shares in one of five event groups (US strikes on Iran, US x Iran ceasefire, Machado Nobel Peace Prize 2025, Maduro out, OpenAI browser) between April 2025 and March 2026. The six features are computed over the wallet's **whole history** in the lab's trade files (1,644 markets, 8.4M trade rows). Every buyer counts, whether it placed the order (maker) or hit one (taker). Column `confirmed` is `True` for the wallets that were later publicly identified as insiders (press reports, an SSRN paper, court filings): our few known positives.

Wallet ids are pseudonymous. `w_` followed by ten hex digits, derived from the real address with a secret salt: the same wallet always gets the same id, and the id cannot be turned back into an address. This course works with pseudonymous wallets only. Do not try to identify the people behind any wallet.

feature	meaning
`position_anomaly_ratio`	biggest single buy divided by the wallet's average spend per market. "Is this bet out of character?"
`market_notional_ratio`	the wallet's largest share of one market's total volume. "Did it move the market?"
`avg_buy_price`	volume-weighted price paid for shares. Insiders buy conviction at 0.2-0.6, not lottery tickets at 0.02
`concentration`	share of all spending that went into one market. All-in = 1
`wallet_age_days`	first to last trade seen. Fresh wallets are cheap to make
`unique_markets`	how many markets the wallet touched

These six are one reasonable set, not the right one. None of them uses the market's outcome: a feature that knows who won has nothing left to detect.

1. Load the wallet table

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
pd.set_option("display.width", 200); pd.set_option("display.max_colwidth", 44)

W = pd.read_csv("data/wallet_features.csv", index_col="wallet")
FEATS = ["position_anomaly_ratio", "market_notional_ratio", "avg_buy_price", "concentration", "wallet_age_days",
        "unique_markets"]
SHORT = {"position_anomaly_ratio": "pos_ratio", "market_notional_ratio": "mkt_share", "avg_buy_price": "price",
        "concentration": "conc",
         "wallet_age_days": "age_d", "unique_markets": "markets", "trade_count": "trades", "total_notional":
        "total_$"}   # short names for display only
print(f"{len(W):,} wallets, {int(W.confirmed.sum())} of them publicly confirmed insiders")
W[FEATS + ["trade_count", "total_notional", "confirmed"]].head(8).rename(columns=SHORT)
```

```
55,952 wallets, 19 of them publicly confirmed insiders
```

wallet	pos_ratio	mkt_share	price	conc	age_d	markets	trades	total_$	confirmed
w_b6640fc6a7	0.365252	0.000322	0.576023	1.000000	1.598889	1	6	241.805798	False
w_ce85723407	1.501651	0.000362	0.936964	0.405506	49.930856	4	6	33.230088	False
w_b92e826e83	1.099906	0.000021	0.340639	0.483274	17.895579	4	7	43.640110	False
w_ab04538147	2.331724	0.000559	0.204462	0.388621	19.121771	6	10	135.093199	False
w_09a7eb8ca6	1.008677	0.000114	0.678542	0.077591	4.868113	13	13	64.902260	False
w_d041e2067b	3.502354	0.000001	0.804635	0.700471	27.958044	5	5	23.369995	False
w_07fdda55e6	1.258649	0.000476	0.923803	0.530658	3.153634	3	8	2951.797788	False
w_aca11e3e0e	2.643086	0.005882	0.957105	0.241208	14.600370	11	24	504.735026	False

2. Stage 1: set the bots aside

Market-making bots trade thousands of times across hundreds of markets. They are the most *anomalous* wallets by any measure, and if we leave them in, the forest spends its attention on them and insiders disappear. So: a wallet with 500+ markets or 2,000+ trades is a bot for our purposes, and is scored 0.

```
W["is_bot"] = (W.unique_markets >= 500) | (W.trade_count >= 2000)
H = W[~W.is_bot].copy()
print(f"bots set aside: {W.is_bot.sum()}   humans to score: {len(H):,}")
```

```
bots set aside: 284   humans to score: 55,668
```

3. Stage 2: fit the Isolation Forest

200 random trees. Each tree isolates points with random axis-parallel cuts; a point that is alone after few cuts is anomalous. `contamination=0.05` tells the model to call the top 5% "outliers". We ignore that binary call and keep the **ranking**.

```
from sklearn.ensemble import IsolationForest
X = H[FEATS].to_numpy(float)
for c in range(X.shape[1]):                          # fill the few missing values with the column median
    bad = ~np.isfinite(X[:, c]); X[bad, c] = np.nanmedian(X[~bad, c])

forest = IsolationForest(n_estimators=200, contamination=0.05, random_state=42, n_jobs=-1).fit(X)
H["raw"] = forest.decision_function(X)               # negative = anomalous
H["score"] = 100 / (1 + np.exp(10 * H.raw))          # 0..100, 100 = most anomalous (the lab's display scale)
H["rank"] = H.raw.rank(method="first").astype(int)   # 1 = most anomalous
H = H.sort_values("rank")
print(f"fitted on {len(H):,} wallets")
```

```
fitted on 55,668 wallets
```

4. The top 1%: who does the forest point at?

```
n = len(H); top1 = int(np.ceil(0.01 * n))
H["insider"] = np.where(H.confirmed, "*** confirmed", "")
print(f"top 1% = {top1} wallets. Confirmed insiders among them: {int(H.iloc[:top1].confirmed.sum())}")
H.iloc[:20][["score", "insider"] + FEATS + ["total_notional"]].round(3).rename(columns=SHORT)
```

```
top 1% = 557 wallets. Confirmed insiders among them: 1
```

wallet	score	insider	pos_ratio	mkt_share	price	conc	age_d	markets	total_$
w_3c83e74381	91.988		19.211	0.176	0.580	0.094	329.206	224	5765.778
w_592a48ad5c	91.827		23.117	0.221	0.655	0.128	330.123	186	28152.600
w_4fad99896f	91.826		23.648	0.169	0.913	0.110	346.541	264	247836.650
w_a085741c51	91.584		36.276	0.288	0.473	0.157	171.848	233	909.161
w_8577bb2d2f	91.114		30.830	0.181	0.932	0.318	274.404	135	12622.713
w_a8a52f1188	90.983		21.672	0.086	0.490	0.099	279.122	221	50936.538
w_cdb76c15a1	90.938		15.124	0.344	0.899	0.179	357.121	137	491129.070
w_420bee7db7	90.663		8.652	0.616	0.966	0.168	326.517	139	316368.892
w_06c7a3672b	90.592		8.517	0.601	0.980	0.061	264.382	157	329191.603
w_cf61d5d09a	90.216		50.003	0.046	0.681	0.238	349.725	218	25303.942
w_5eb96fcdc1	89.938		7.243	0.638	0.989	0.283	352.180	101	557204.956
w_ce1f7a39ef	89.913		5.013	0.433	0.918	0.091	300.100	227	226420.182
w_5543fb0846	89.879		12.659	0.119	0.604	0.175	328.921	154	164711.801
w_0cdb25d1eb	89.866		6.680	0.515	0.953	0.251	320.797	147	488536.198
w_84a3dd05ec	89.806		9.510	0.361	0.752	0.118	327.368	144	90666.818
w_67ea7d6162	89.784		1.768	0.694	0.930	0.046	328.287	161	265703.636
w_8c69cfa6f4	89.772		25.811	0.921	0.904	0.534	238.965	52	147164.533
w_23adb00dbd	89.759		66.392	0.017	0.889	0.419	327.541	264	10915.293
w_47caff27bd	89.706		8.319	0.295	0.960	0.071	276.037	151	126934.822
w_abc95bcad5	89.545		8.823	0.133	0.665	0.067	314.417	256	89270.311

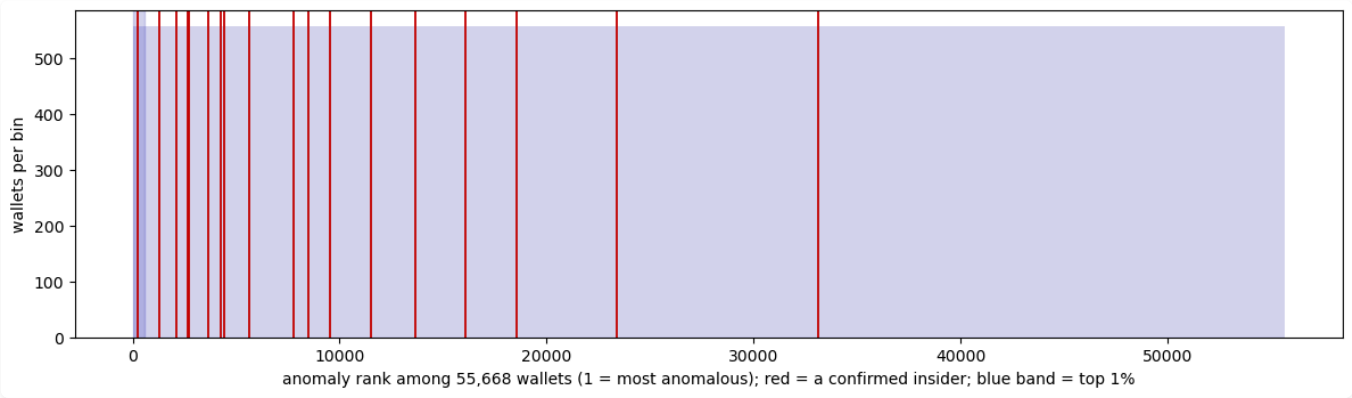
5. Where do the confirmed insiders land?

This is the only evaluation we can do with a handful of positive labels: **ranks**, not accuracy.

```
lab = H[H.confirmed]
lab[["rank", "score"] + FEATS + ["total_notional"]].assign(percentile=(lab["rank"] / n *
        100).round(2)).round(3).rename(columns=SHORT)
```

wallet	rank	score	pos_ratio	mkt_share	price	conc	age_d	markets	total_$	percentile
w_2a43266bd8	264	81.759	4.063	0.261	0.897	0.147	300.514	55	675486.924	0.47
w_bc876cca82	1301	63.735	0.607	0.151	0.788	0.251	41.889	22	359435.595	2.34
w_bae5b984cf	2109	54.765	1.575	0.093	0.562	0.291	39.846	13	297220.768	3.79
w_b484b941ed	2665	50.727	4.645	0.011	0.661	0.579	244.999	26	489359.213	4.79
w_651e7829e2	2741	50.277	0.756	0.034	0.815	0.302	241.940	8	105116.362	4.92
w_64ac068f22	3670	45.168	1.056	0.013	0.450	0.412	260.358	10	26935.346	6.59
w_d1a074f023	4238	42.609	0.773	0.026	0.727	0.415	208.946	6	65325.714	7.61
w_ce629884a4	4400	42.041	2.093	0.006	0.734	0.108	120.232	38	3811.868	7.90
w_45933e3f6b	5647	38.174	0.450	0.040	0.533	0.577	9.532	3	42668.076	10.14
w_399ef66fe6	7801	33.051	3.089	0.019	0.827	0.803	8.289	4	16209.535	14.01
w_b61c7eb379	8480	31.901	0.693	0.004	0.152	0.694	8.676	2	5233.982	15.23
w_2023847f0c	9559	30.285	0.333	0.000	0.200	1.000	0.019	1	18136.204	17.17
w_2c4d5f68c4	11539	28.135	0.827	0.003	0.234	0.747	0.049	2	14392.083	20.73
w_efad88a8e8	13682	26.251	0.600	0.000	0.100	1.000	0.000	1	1865.159	24.58
w_cd38860893	16055	24.497	0.210	0.004	0.585	0.706	2.252	2	75327.419	28.84
w_ea3af901bf	18532	22.905	2.758	0.000	0.416	0.813	11.461	7	10786.728	33.29
w_bf68f8e7eb	23399	20.393	0.721	0.002	0.822	0.198	5.685	9	46028.969	42.03
w_89b41a54ca	33136	17.112	1.539	0.001	0.793	0.212	7.803	10	38444.569	59.52

```
fig, ax = plt.subplots(figsize=(12, 3.6))
ax.hist(H["rank"], bins=100, color="#d2d2ef")
for _, r in lab.iterrows():
    ax.axvline(r["rank"], color="#c40000", lw=1.3)
ax.axvspan(0, top1, color="#3333b2", alpha=0.2)
ax.set_xlabel(f"anomaly rank among {n:,} wallets (1 = most anomalous); red = a confirmed insider; blue band =
        top 1%")
ax.set_ylabel("wallets per bin"); plt.tight_layout(); plt.show()
```



6. Precision at k, against chance and against a simple baseline

If we flag the top k% and hand the list to a human, how many confirmed insiders are on it? Compare with chance, and with the simplest possible ranking: **biggest spenders first**. Always run a simple baseline; if your model does not beat it, say so.

```
H["spend_rank"] = H.total_notional.rank(ascending=False, method="first")     # the simple baseline: rank by
        total spend
rows = []
for k in (0.5, 1, 2.5, 5, 10):
    cut = int(np.ceil(k / 100 * n))
    rows.append(dict(top_percent=k, wallets_flagged=cut, of=len(lab),
                     forest_catches=int((lab["rank"] <= cut).sum()),
                     biggest_spenders_catch=int((H.loc[lab.index, "spend_rank"] <= cut).sum()),
                     chance=round(len(lab) * k / 100, 2)))
pd.DataFrame(rows)
```

	top_percent	wallets_flagged	of	forest_catches	biggest_spenders_catch	chance
0	0.5	279	18	1	4	0.09
1	1.0	557	18	1	5	0.18
2	2.5	1392	18	2	9	0.45
3	5.0	2784	18	5	13	0.90
4	10.0	5567	18	8	16	1.80

The biggest-spenders ranking wins, and that is the lesson, not a failure: the *confirmed* insiders are the ones the press caught, and the press caught the biggest winners. Size finds them. The forest ranks by how **unusual** a wallet is, which is a different question, and the insiders that neither ranking finds are the small, fresh ones (section 9).

7. One insider vs one ordinary trader

The highest-ranked confirmed insider next to a wallet from the middle of the ranking with a normal footprint.

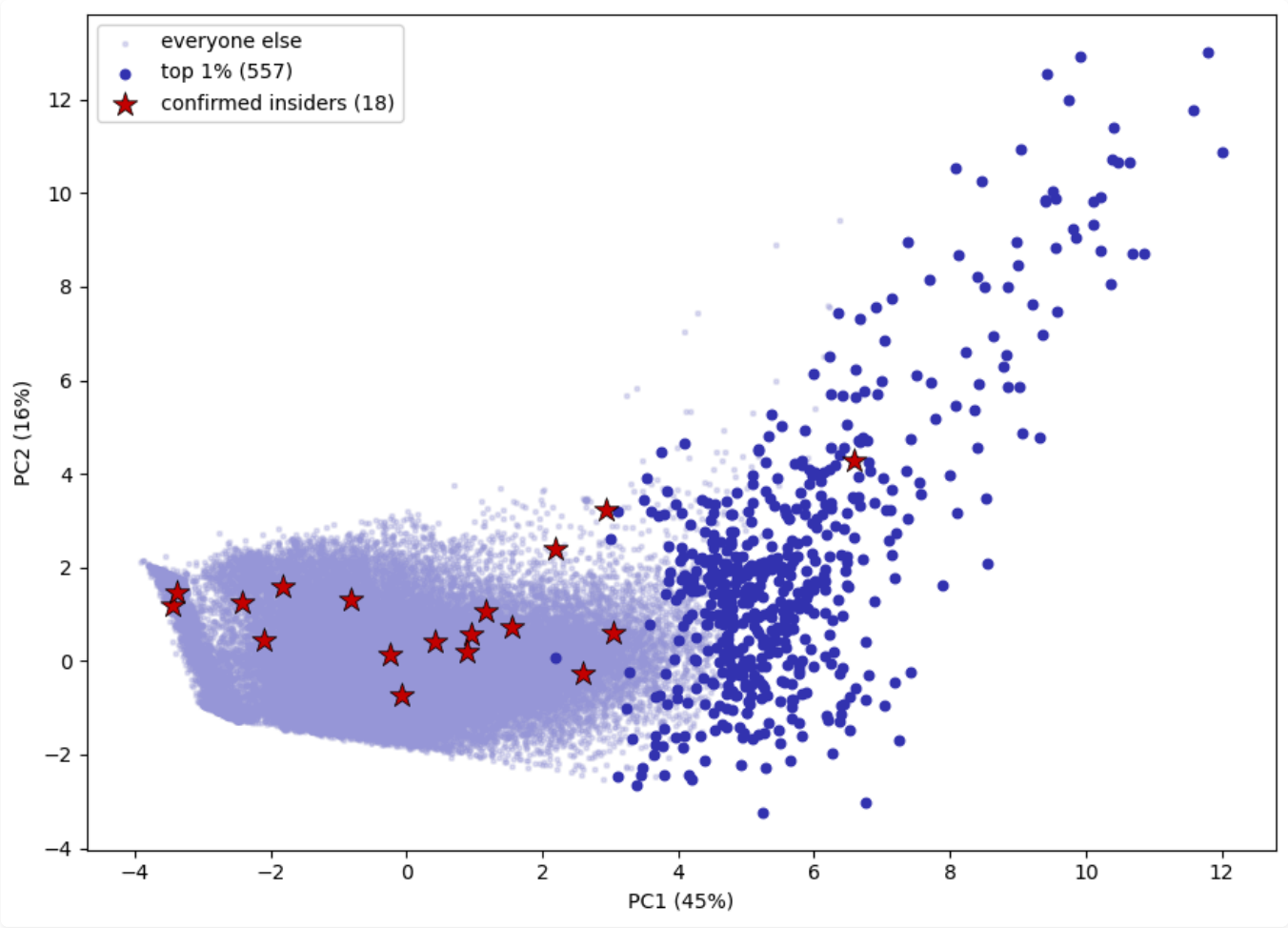
```
best = lab.iloc[0]
mid = H.iloc[n // 2 - 50: n // 2 + 50]
ordinary = mid[(mid.trade_count >= 10) & (mid.unique_markets >= 2)].iloc[0]
pd.DataFrame({f"confirmed insider ({best.name})": best, f"ordinary trader ({ordinary.name})":
        ordinary}).loc[["rank", "score"] + FEATS + ["trade_count", "total_notional"]].round(3)
```

	confirmed insider (w_2a43266bd8)	ordinary trader (w_8379b038ff)
rank	264	27787
score	81.758529	18.762895
position_anomaly_ratio	4.062995	2.058684
market_notional_ratio	0.260756	0.000407
avg_buy_price	0.896895	0.932057
concentration	0.146908	0.182673
wallet_age_days	300.513912	19.12287
unique_markets	55	12
trade_count	1332	23
total_notional	675486.923909	306.35538

8. The whole population in two dimensions (PCA, from Week 1)

Standardise the log of the six features, project to two components. Blue = flagged by the forest, red stars = confirmed insiders.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
Xs = H[FEATS].to_numpy(float)                        # H is sorted by rank now, so rebuild the matrix in that
        order
for c in range(Xs.shape[1]):
    bad = ~np.isfinite(Xs[:, c]); Xs[bad, c] = np.nanmedian(Xs[~bad, c])
Z = StandardScaler().fit_transform(np.log1p(np.clip(Xs, 0, None)))
P = PCA(2, random_state=0).fit(Z); pc = P.transform(Z)
flag = (H["rank"] <= top1).to_numpy(); li = H.confirmed.to_numpy()
fig, ax = plt.subplots(figsize=(9, 6.5))
ax.scatter(pc[~flag, 0], pc[~flag, 1], s=5, color="#9a9ad8", alpha=0.3, label="everyone else")
ax.scatter(pc[flag, 0], pc[flag, 1], s=22, color="#3333b2", label=f"top 1% ({top1})")
ax.scatter(pc[li, 0], pc[li, 1], s=160, marker="*", color="#c40000", edgecolor="k", lw=0.5, label=f"confirmed
        insiders ({li.sum()})")
ax.set_xlabel(f"PC1 ({P.explained_variance_ratio_[0]:.0%})"); ax.set_ylabel(f"PC2
        ({P.explained_variance_ratio_[1]:.0%})"); ax.legend()
plt.tight_layout(); plt.show()
```



9. What the misses have in common

The confirmed insiders the forest did *not* flag. Look at `unique_markets`, `wallet_age_days` and `trade_count`.

```
missed = lab[lab["rank"] > int(np.ceil(0.05 * n))]
missed[["rank", "unique_markets", "wallet_age_days", "trade_count", "concentration",
        "total_notional"]].round(2).rename(columns=SHORT)
```

wallet	rank	markets	age_d	trades	conc	total_$
w_64ac068f22	3670	10	260.36	96	0.41	26935.35
w_d1a074f023	4238	6	208.95	199	0.42	65325.71
w_ce629884a4	4400	38	120.23	310	0.11	3811.87
w_45933e3f6b	5647	3	9.53	136	0.58	42668.08
w_399ef66fe6	7801	4	8.29	31	0.80	16209.54
w_b61c7eb379	8480	2	8.68	81	0.69	5233.98
w_2023847f0c	9559	1	0.02	122	1.00	18136.20
w_2c4d5f68c4	11539	2	0.05	75	0.75	14392.08
w_efad88a8e8	13682	1	0.00	7	1.00	1865.16
w_cd38860893	16055	2	2.25	155	0.71	75327.42
w_ea3af901bf	18532	7	11.46	193	0.81	10786.73
w_bf68f8e7eb	23399	9	5.69	134	0.20	46028.97
w_89b41a54ca	33136	10	7.80	139	0.21	38444.57

A brand-new wallet that makes a handful of bets in one market and never trades again is **not anomalous**: thousands of one-shot gamblers look exactly like that. Per-wallet features cannot separate a burner insider from a burner gambler. What could is context that one wallet's rows do not carry: when the bets happened relative to the news, and what moved in the related markets of the same event. That is where the lecture ends, and where your project could begin.

10. Save your ranking

One line. The file lands next to the wallet table; open it in Excel or hand it to a teammate.

```
H.drop(columns=["insider", "spend_rank"]).to_csv("data/wallet_scores.csv")
print("saved data/wallet_scores.csv with", len(H), "rows")
```

```
saved data/wallet_scores.csv with 55668 rows
```

```
```